

GuestEase

A full-stack web app for small hospitality businesses

Student Name: Andrea Nardinocchi

Student Number: 20108884

Programme: Higher Diploma in Science in Computer Science 2024

Project Type: Independent Project

Project Category: Web Application



Project Overview

GuestEase is a prototype full-stack booking platform app for an **8-room B&B**, with 2 user roles: '**guest**' and '**admin**'.

It features full booking **CRUD functionalities** for both user roles, Guest + Admin dashboards, Stripe payments processing, email notifications, and guest reviews.



The Problem

Market Analysis: Digital Gap in Irish Small Accommodation Sector

- Many Irish B&Bs and guesthouses still rely on **manual bookings & outdated websites**
- Yet **80%** of **reservations** are now **online**, so **slow digital adoption** creates inefficiencies and fails to meet guest expectations

Guest Behaviour is Rapidly Shifting

- 59% of bookings via smartphones (2025)
- +15% growth in short-term rental platforms in 2024
- +8.2% increase in domestic stays
- Hotel occupancy above pre-pandemic levels

Key Challenges

- Manual systems => inefficiency & double bookings
- No 24/7 booking => lost revenue
- OTAs reliance => 15-30% commission + limited data
- Weak digital presence → lost competitiveness

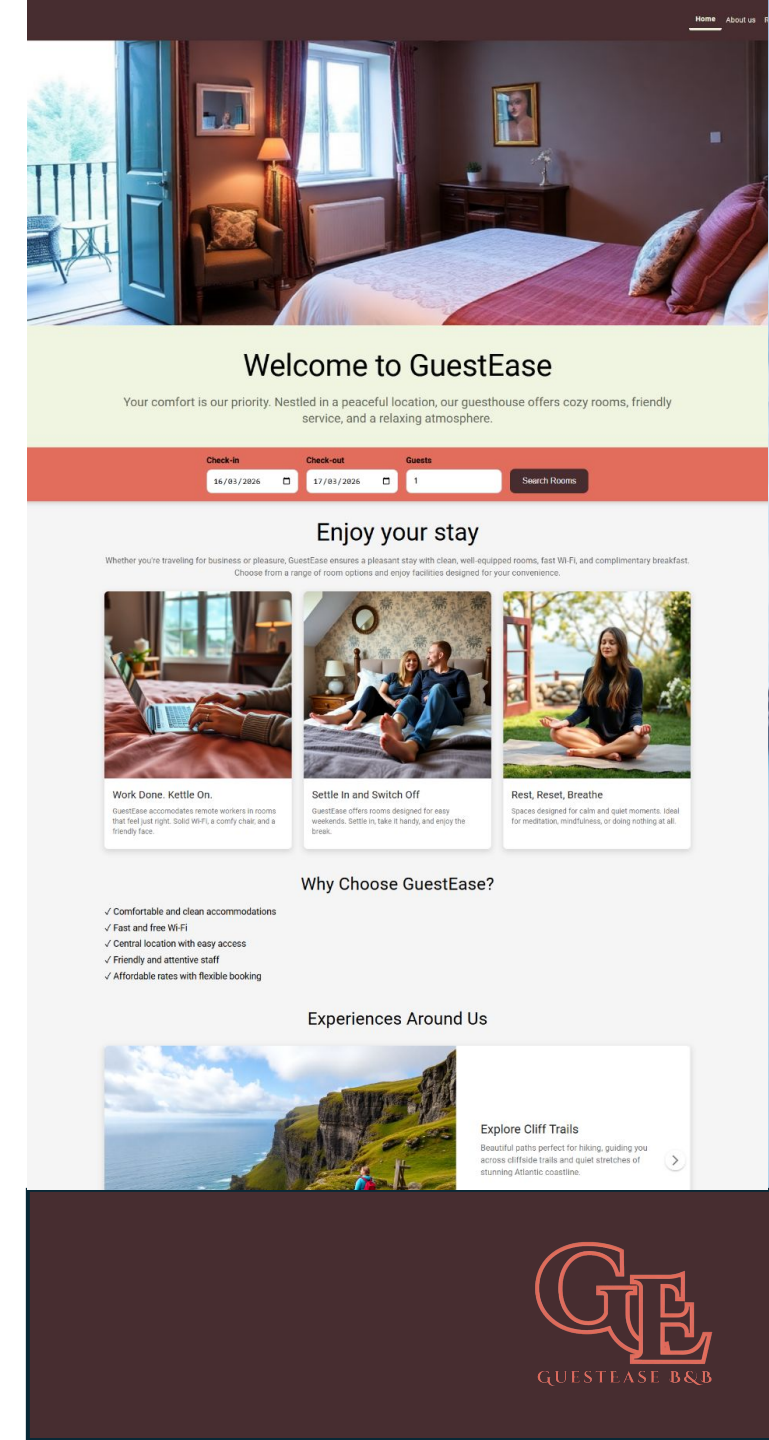
Insight

Digital adoption is no longer optional — it's critical to compete



The Solution: GuestEase

- Direct control of room pricing, bookings & guest data (no OTAs commissions) - Admin
- Direct online bookings with real-time availability & secure payments - Guest
- Automated emails & workflows reduce admin workload
- Mobile-friendly experience improves guest satisfaction & competitiveness



Live Demo

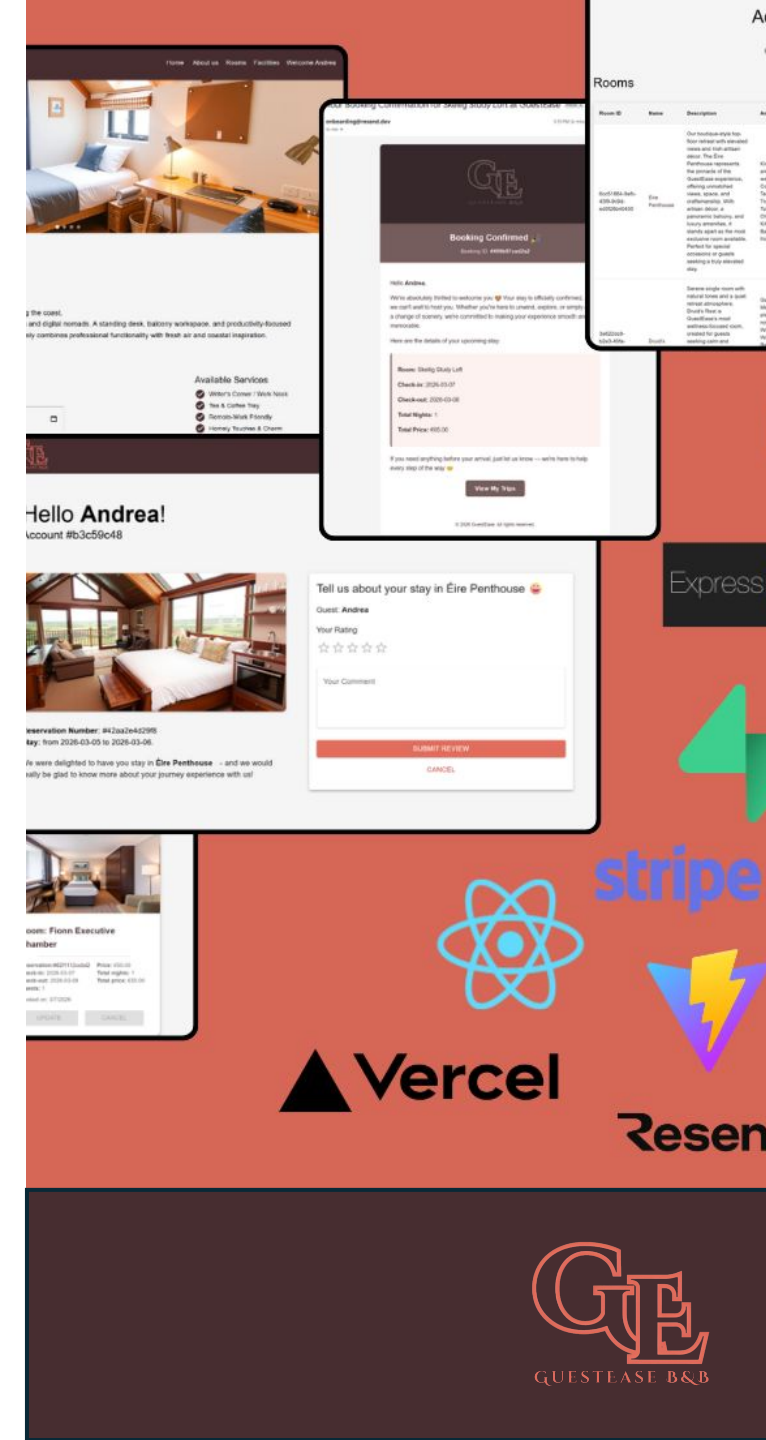
Backend: <https://guestease.onrender.com/>

Frontend: <https://guest-ease-efpg.vercel.app/>

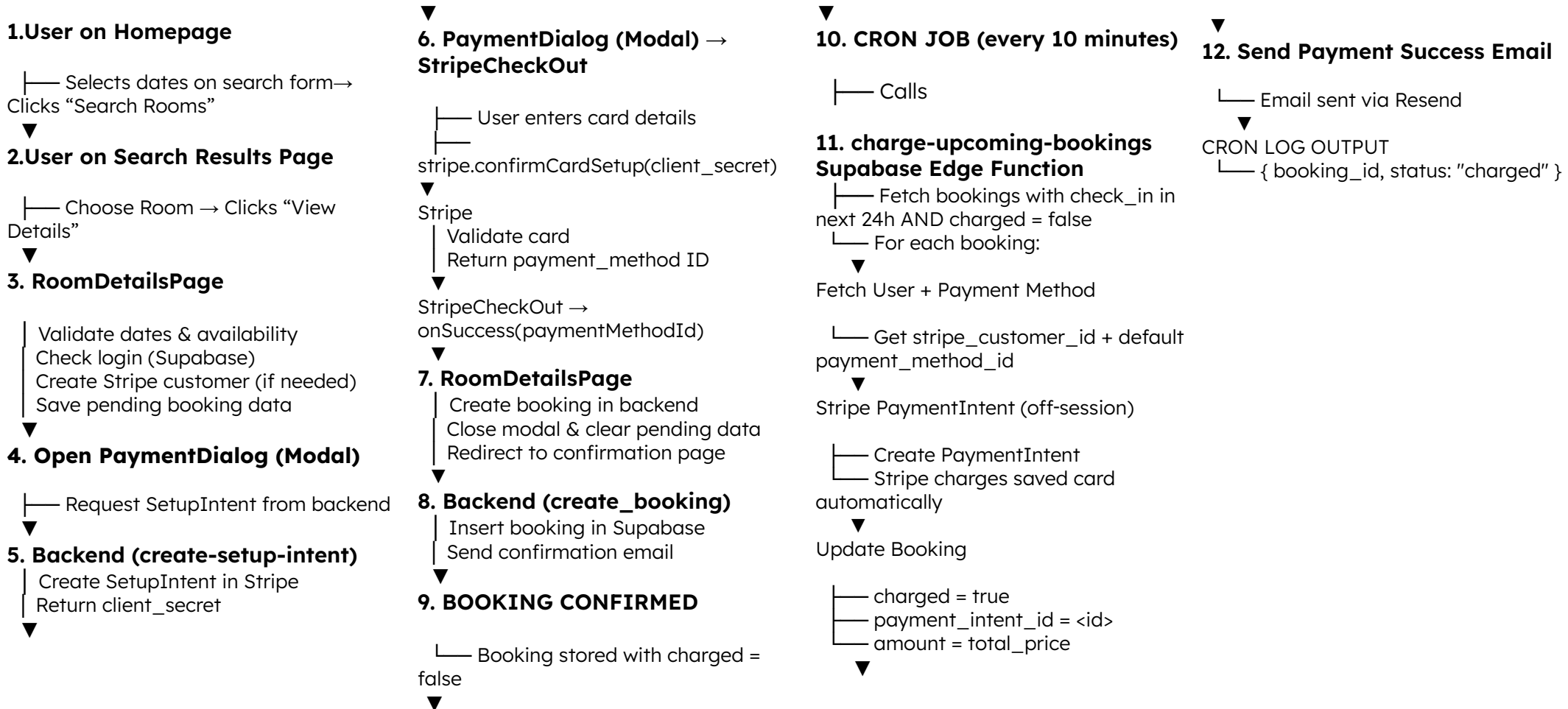
- Guest booking flow
- Admin dashboard walkthrough

Backup Video:

<https://www.youtube.com/watch?v=2RxNIrUVv5Q&t=3s>

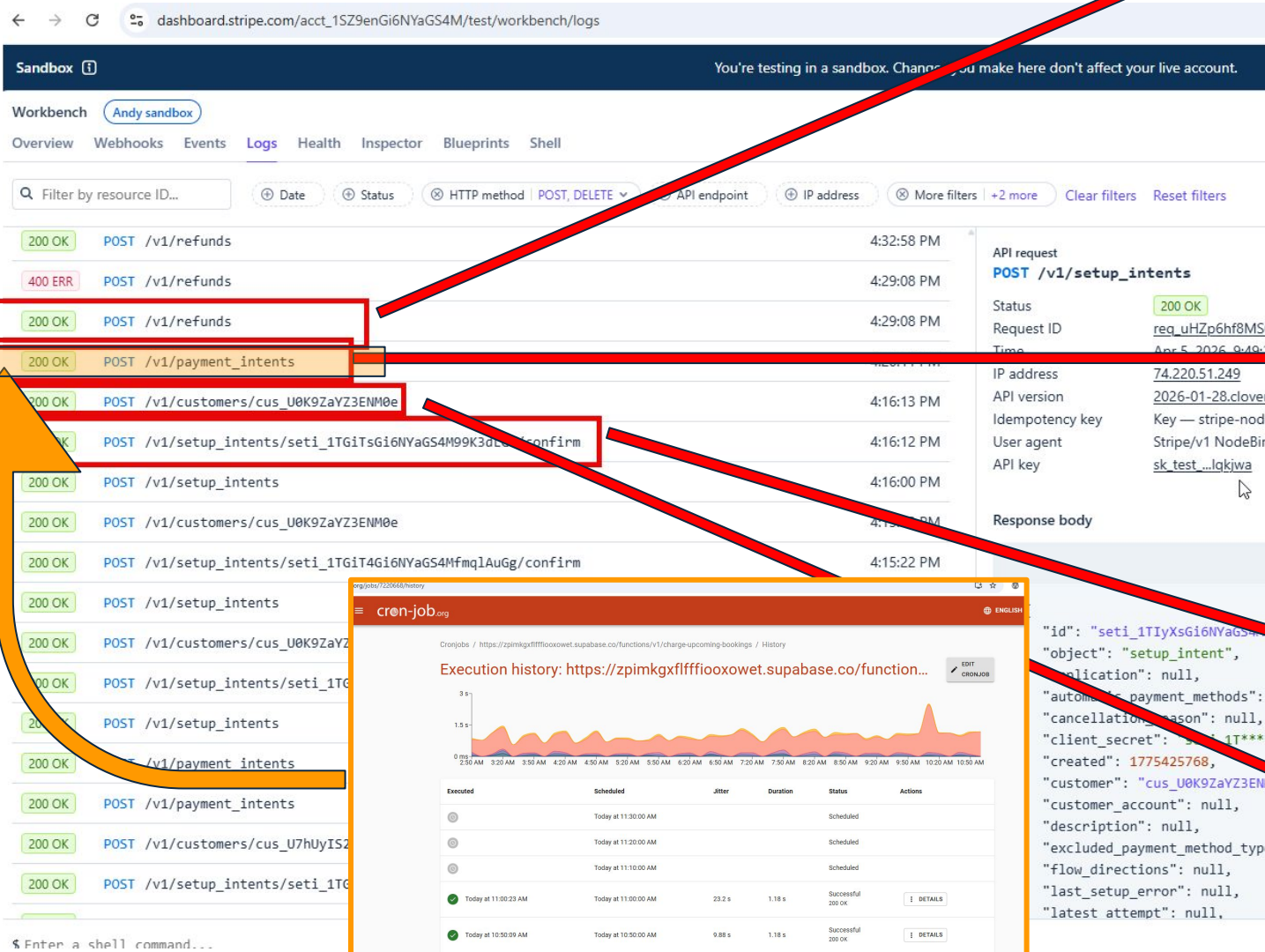


Guest booking/payment flow highlight



Stripe dashboard logs & Cron

https://dashboard.stripe.com/acct_1SZ9enGi6NYaGS4M/test/workbench/logs



<https://console.cron-job.org/jobs/7220668/history>

```
// https://docs.stripe.com/api/refunds/create?lang=node
if (amountToRefund > 0) {
  await stripe.refunds.create({
    payment_intent: booking.payment_intent_id,
    amount: amountToRefund,
  });
} catch (refundErr) {
```

backend/routes/adminCancelBookings.js

```

    * https://docs.stripe.com/api/payment\_intents/create
    * Convert total_price (string) to number
    */
    const paymentIntent = await stripe.paymentIntents.create(
      {
        amount: Number(booking.total_price) * 100,
        currency: "eur",
        customer: user.stripe_customer_id,
        payment_method: pm.payment_method_id,
        // 'confirm: true' is used to immediately charge the payment method.
        // https://stripe.com/docs/payments/payment-intents/quickstart#confirm
        // https://docs.stripe.com/payments/accept-a-payment-synchronously
        confirm: true,
        // 'off session: true' tells Stripe the customer is not present, allowing
        // background/automatic charges without 3D Secure prompts.
        // This is needed for cron https://console.cron-job.org/jobs/7220668/history
        // https://support.stripe.com/questions/what-is-the-difference-between-on-session-and-off-session
        off_session: true,
        description: `Booking ${booking.id} charge`,
      },
    ),
  {
    // Ensures the same booking cannot be charged twice
    // https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Idempotency-Key
    idempotencyKey: booking.id,
  }
);

```

Supabase Edge Functions/charge-upcoming-bookings

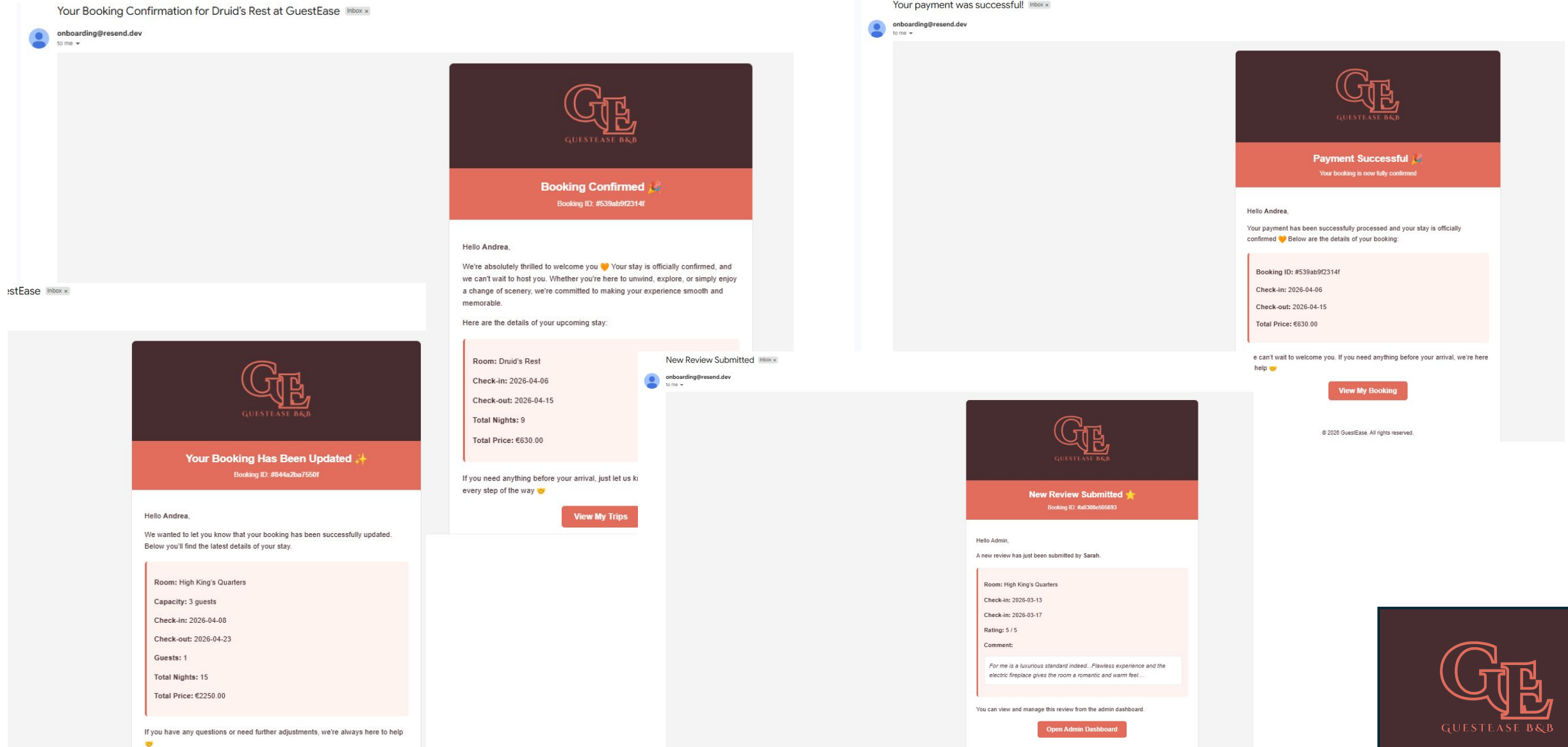
```
const setupIntent = await stripe.setupIntents.create({
  customer: customerId,
  payment_method_types: ["card"],
});
```

```
backend/routes/stripeSetupIntentPayments.js
```

```
* https://docs.stripe.com/api/customers/create
*/
const customer = await stripe.customers.create({
  email,
  metadata: { userId },
});
```

```
backend/routes/createStripeCustomer.js
```

Email notifications examples



System Architecture

Frontend

- React + TypeScript (Vite)
- Material UI
- React Router & React Query

Backend

- Node.js (Express)
- Supabase Server Client
- Supabase Edge Functions

Database & Auth

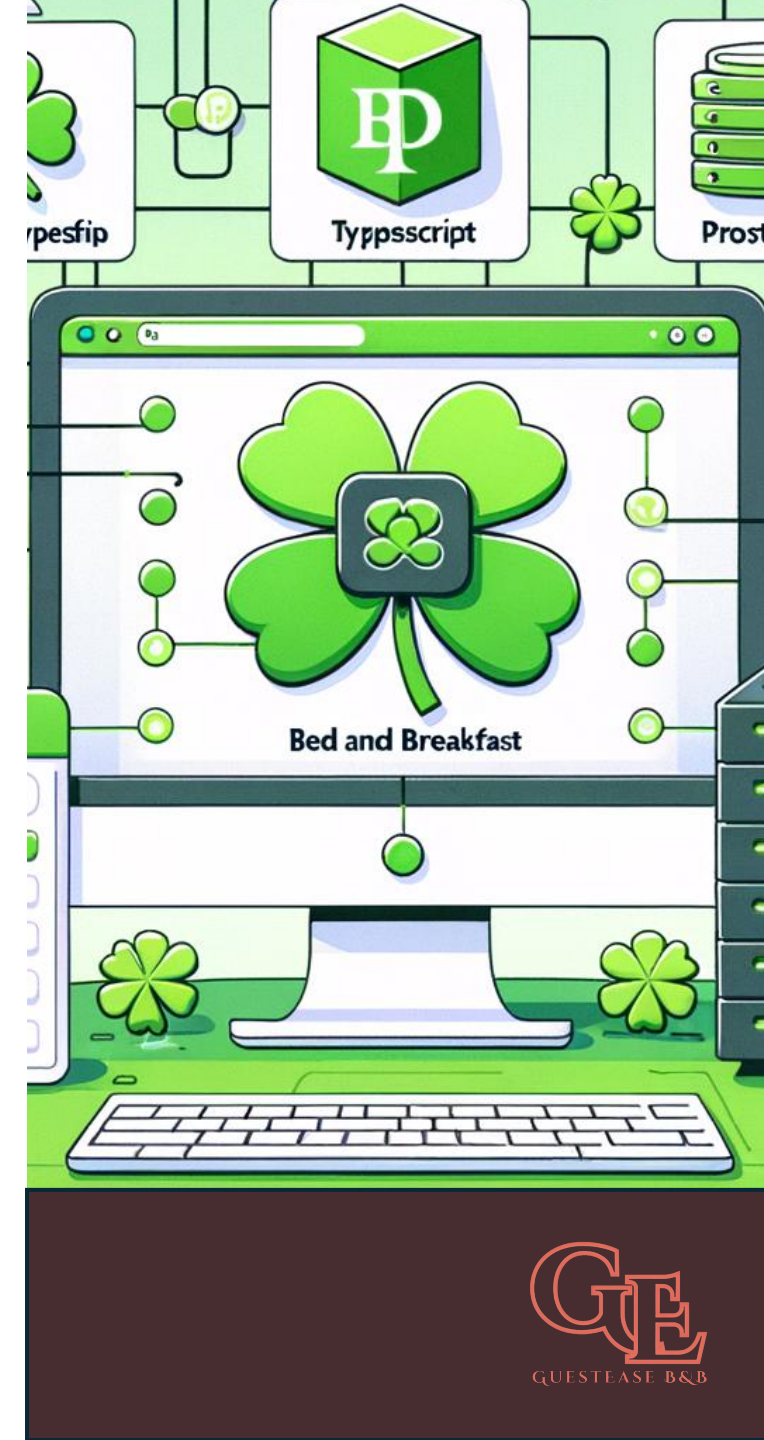
- Supabase PostgreSQL
- Supabase Auth + RLS (Row-Level Security)

Services & Integrations

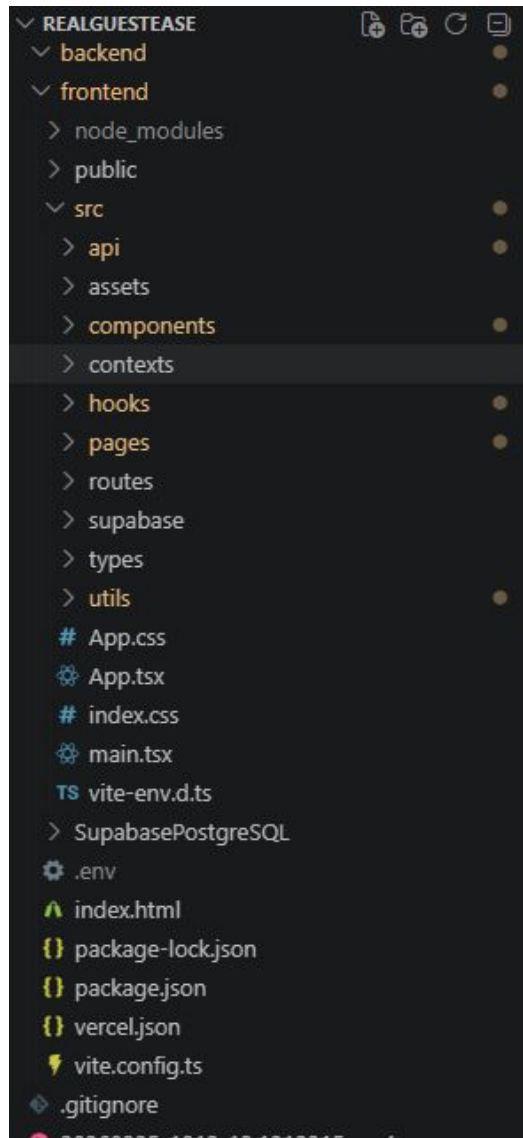
- Stripe (payments)
- Resend (emails)
- Cron-job.org (scheduled tasks)

Storage & Deployment

- Supabase Storage
- Vercel (frontend)
- Render (backend)

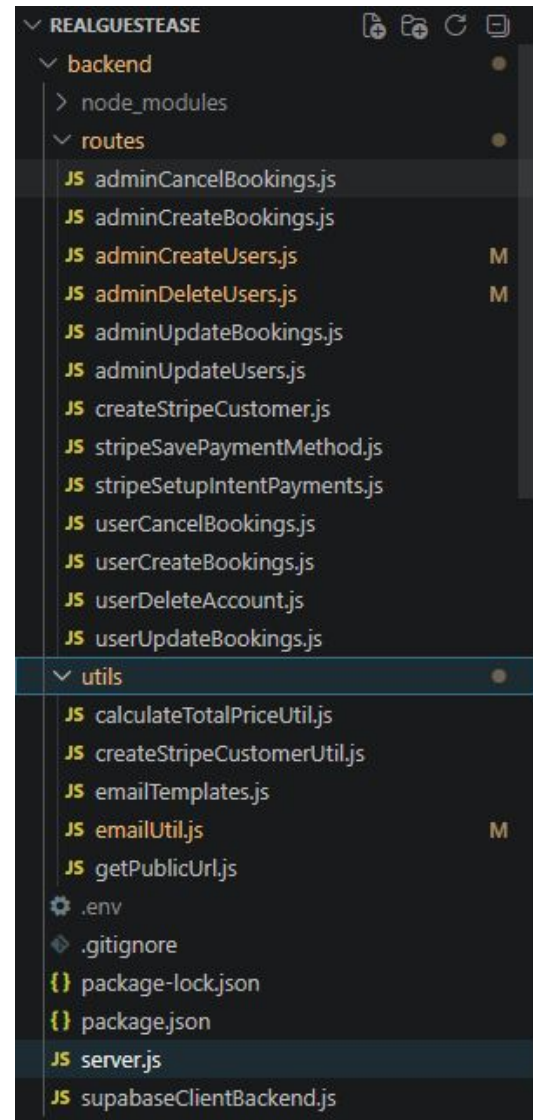


System Architecture



Frontend

A clear folder structure is created to implement a separation of concerns between states and data logic: components, pages, hooks, contexts, API, routes, types, and utils plus a Supabase util folder..



Backend

The backend presents a clear Express router structure with separate utility keeping the code organized and scalable.

<https://github.com/AndreaNardinocchi/GuestEase>

Analysis & Design

Frontend Technology Analysis

Framework	Suitability	Learning Curve	Ecosystem	Performance	Confidence / Maintenance	Notes	Grade
React	High	Moderate	Very large	High	High - already used	Used by Marriott.com	A 
Svelte	Medium	Moderate	Small	Very High	Medium - less confident	Growing framework	B
Angular	Low	Steep	Large	Medium	Low - unfamiliar	Enterprise-focused	C-
Vue.js	Low	Gentle	Moderate	High	Low - no experience	Less enterprise adoption	C



Analysis & Design

Backend/Database Technology Analysis

Framework	Suitability	Learning Curve	Ecosystem	Performance	Confidence / Maintenance	Notes	Grade
Node.js	High	Moderate	Very large	High	High - already used	Flexible, requires DB setup	B+ 
Firebase	Medium	Gentle	Large	Medium	Medium - some NoSQL experience	Fast BaaS, limited relational support	C+
Supabase	High	Moderate	Medium	High	High - already used	PostgreSQL, real-time, relational queries	A 



Analysis & Design

Repos Technology Analysis

Project	Purpose	Front-End	Back-End / Server	Database	Authentication / Admin / Notable Tech
Hotel Booking System (Othmane Nissoukin) 	Full hotel booking system with customer-facing and admin interfaces	Next.js (React, Tailwind CSS), React Leaflet, Date-fns, DatePicker, Zod	Next.js API routes	Supabase (PostgreSQL)	Supabase Auth (email, Google, Facebook), user dashboard, admin/back-office dashboards for managing rooms, bookings, and occupancy
Full-stack-hot el-booking (Suresh K)	Full-stack hotel booking with user management	React, Bootstrap, Axios	Node.js + Express	MongoDB	JWT authentication, admin dashboard, role-based access
The-Fern-Hotel-Booking-System (Satish Surani)	Customer-facing hotel booking	HTML, CSS (Bootstrap), JavaScript/AJAX	PHP	MySQL	Admin panel, email notifications, AJAX-based real-time availability



Analysis & Design

Competitor UX Comparison*

Platform	Strengths	Weaknesses	Overall UX
Marriott International	Sticky header + CTA, intuitive language selector, map + filters, strong account navigation	Few usability issues	★ Best
Hilton Worldwide	Map view, clear hotel cards, good booking visibility	Header/form not sticky, weak account guidance	⚖ Mixed
IHG Hotels & Resorts	Functional booking flow, amenities shown	No map, disruptive navigation, less intuitive language selection	⚠ Weak

*Table created using AI-assisted summarization (Microsoft Copilot).

GuestEase Takeaways

- ✓ **Sticky navigation + clear booking CTA** => always visible, seamless flow
- ✓ **Map integration (future feature)** => enhance search experience & hotel discovery
- ✓ **Flexible dates (future enhancement)** => improve user convenience and search options
- ✓ **Simple and easy-to-navigate UI interface** => easy booking and clear account navigation
- ✓ **Mobile-first design** => responsive, fast, and user-friendly



Analysis & Design

Modelling & Design – GuestEase

Users

- **Guest:** books, manages booking, manages profile, leaves reviews
- **Admin:** manages rooms, bookings, users, and reads reviews

Core Data Model

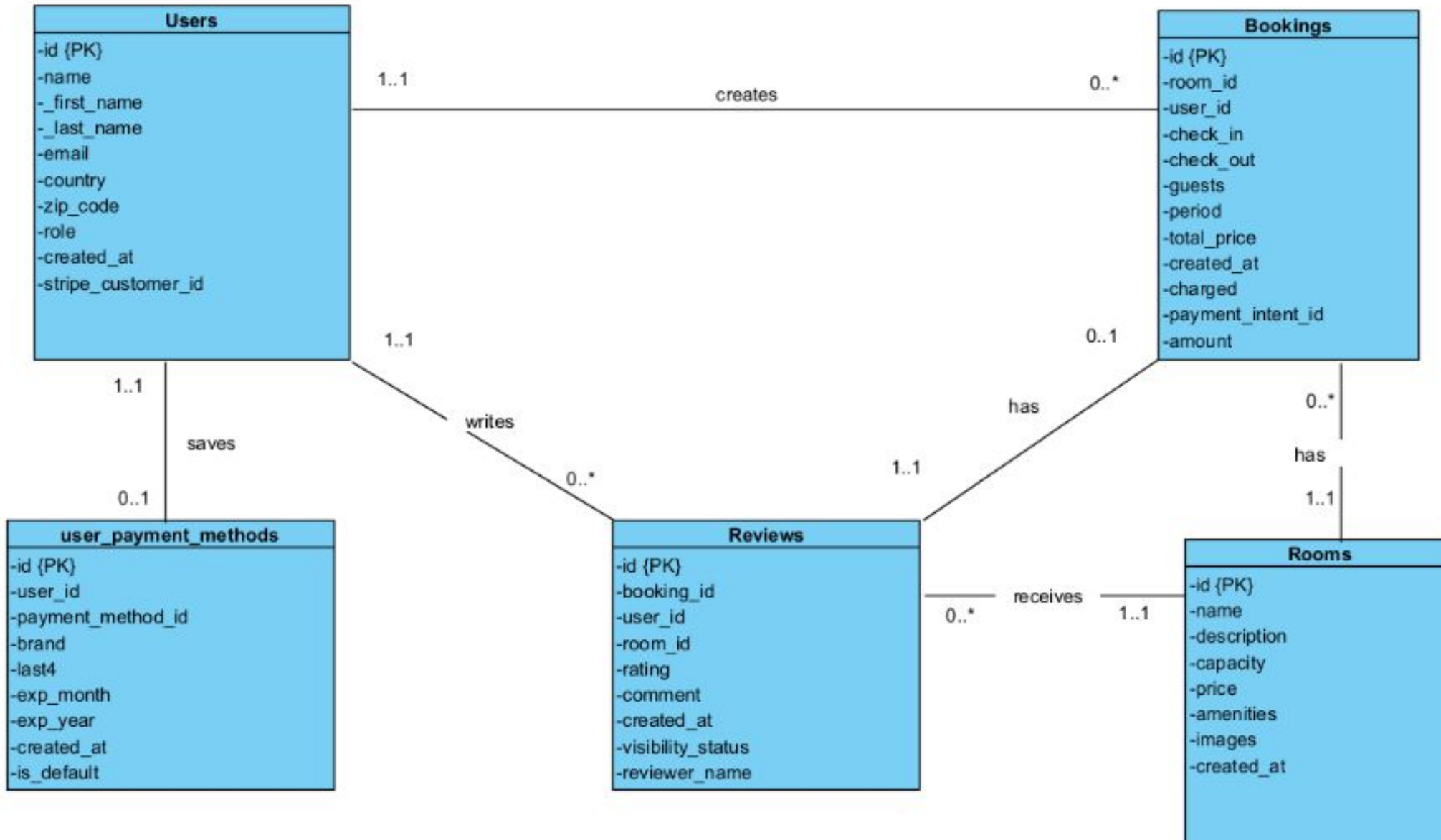
- **Users** => stores guest/admin information/data
- **Rooms** => room details and pricing
- **Bookings** => reservation data
- **user_payment_methods** => stores payment details
- **Reviews** => guest feedback

Core Relationships

- User(Guest/Admin) 1..1 creates 0..* Bookings
- User(Guest) 1..1 saves 0..1 PaymentMethod (user_payment_methods)
- Room 1..1 has 0..* Bookings
- Booking 1..1 generates 0..1 Review
- User(Guest) 1..1 writes 0..* Reviews
- Room 1..1 receives 0..* Reviews
- Booking 1..1 has 1 Stripe PaymentIntent (external)
- Booking 1..1 triggers 0..* Email Notifications (Conceptual Only)
- User(Guest/Administrator) 1..1 receives 0..* Email Notifications (Conceptual Only)



Logical/Physical Data Model (ERD)



Supabase database 'public' schema

```
GuestEase / main PRODUCTION Connect
bookings Multiuse updates rooms function get...
-- ON DELETE CASCADE ensures profiles are removed when users are deleted
11 CREATE TABLE IF NOT EXISTS public.profiles (
12   id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
13   email text NOT NULL UNIQUE,
14   first_name text NOT NULL,
15   last_name text NOT NULL,
16   country text NOT NULL,
17   zip_code text NOT NULL,
18   role text DEFAULT 'guest',
19   created_at timestamptz DEFAULT now()
20 );
21 -- life inside Stripe
alter table profiles
add column stripe_customer_id text;
```

public.profiles table

```
GuestEase / main PRODUCTION Connect
bookings Multiuse updates rooms function get_available_rooms
7 CREATE TABLE IF NOT EXISTS public.bookings (
8   -- https://www.postgresql.org/docs/current/functions-uuid.html
9   id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
10  room_id uuid NOT NULL,
11  user_id uuid NOT NULL,
12  check_in date NOT NULL,
13  check_out date NOT NULL,
14  guests integer NOT NULL,
15  -- Automatically generated daterange used for overlap detection
16  -- '[' = inclusive lower bound, exclusive upper bound
17  -- https://www.postgresql.org/docs/current/rangetypes.html
18  period daterange GENERATED ALWAYS AS (daterange(check_in, check_out, '[')) STORED,
19  total_price numeric,
20  created_at timestamptz DEFAULT now()
21 );
alter table bookings
add column charged boolean default false;
ALTER TABLE bookings
ADD COLUMN payment_intent_id text;
ALTER TABLE bookings
ADD COLUMN amount numeric;
-- We wanted to be able to delete the user having no active reservations
-- We did so by allowing the user_id field to be NULL
ALTER TABLE public.bookings
ALTER COLUMN user_id DROP NOT NULL;
```

public.bookings table

```
GuestEase / main PRODUCTION Connect
bookings Multiuse updates rooms function g
-- Create rooms table with images field
1 CREATE TABLE IF NOT EXISTS public.rooms (
2   -- We ensure that each room has its own id
3   id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
4   name text NOT NULL,
5   description text,
6   capacity integer NOT NULL, -- number of guests (max)
7   price numeric(10,2) NOT NULL,
8   amenities jsonb, -- These will be tags that I can filter off of
9   images jsonb,
10  created_at timestamptz DEFAULT now()
11 );
```

public.rooms table

```
GuestEase / main PRODUCTION Connect
bookings Multiuse updates rooms function get_av
1 DROP TABLE IF EXISTS public.reviews;
2 CREATE TABLE IF NOT EXISTS reviews (
3   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4   booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
5   user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
6   room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
7   rating INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),
8   comment TEXT NOT NULL,
9   created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
10  visibility_status TEXT DEFAULT 'visible',
11  CONSTRAINT one_review_per_booking UNIQUE (booking_id)
12 );
13 -- Store the reviewer's display name directly on each review so
alter table reviews add column reviewer_name text;
```

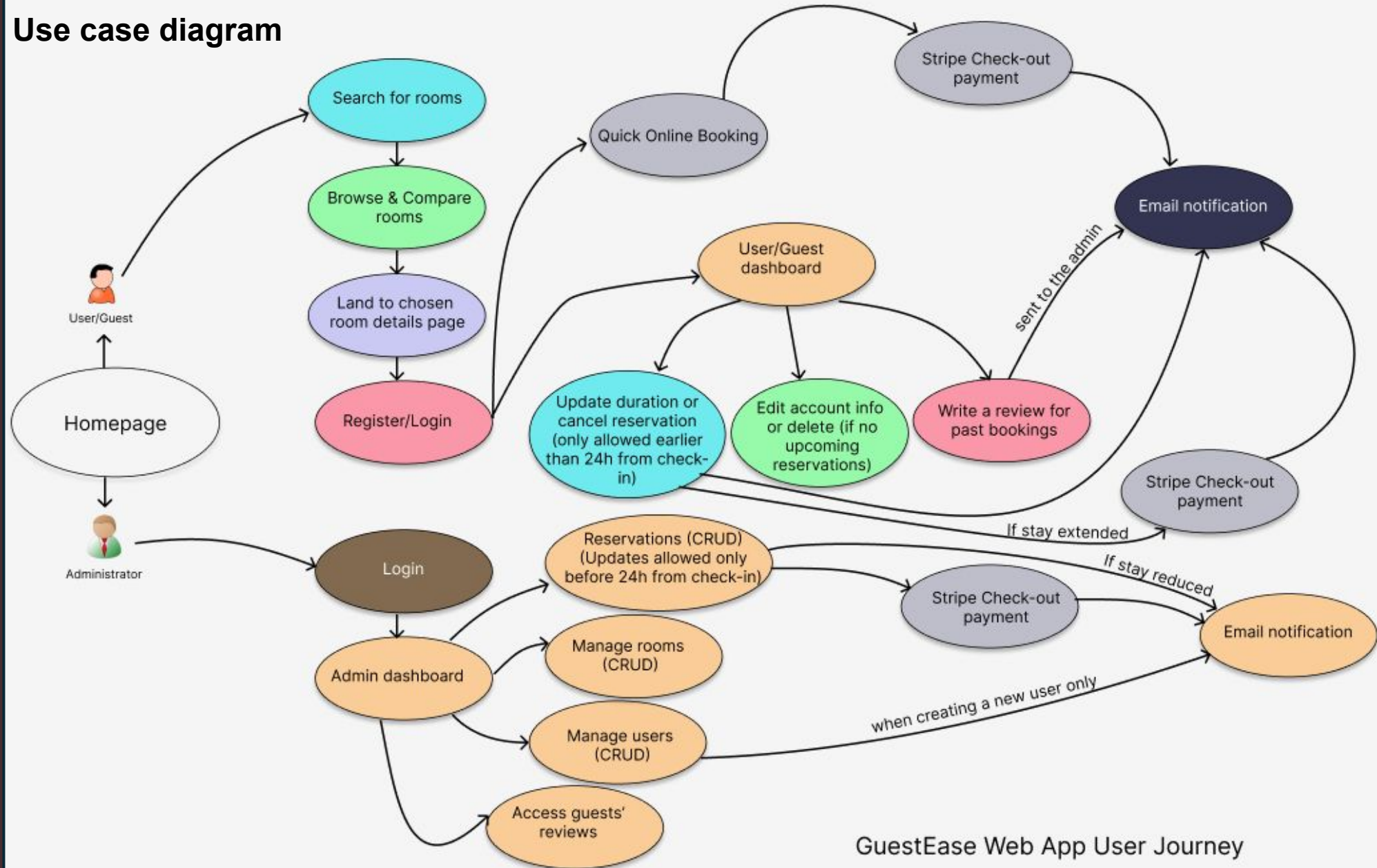
public.reviews table

```
GuestEase / main PRODUCTION Connect
bookings Multiuse updates rooms function get_available_rooms
5 -- https://docs.stripe.com/api/payment_methods/object#payment_method_object-card
6 create table user_payment_methods (
7   id uuid primary key default gen_random_uuid(),
8   user_id uuid references auth.users(id) on delete cascade,
9   payment_method_id text not null,
10  brand text,
11  last4 text,
12  exp_month integer,
13  exp_year integer,
14  is_default boolean default false,
15  created_at timestamp default now()
16 );
```

public.user_payment_methods table

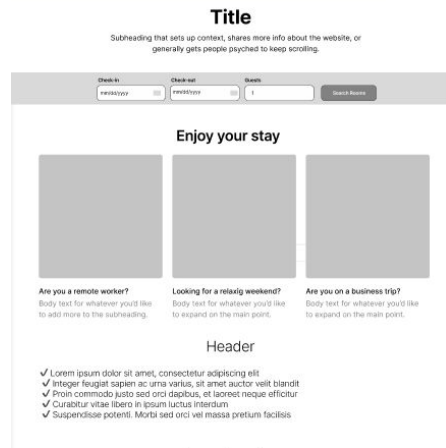
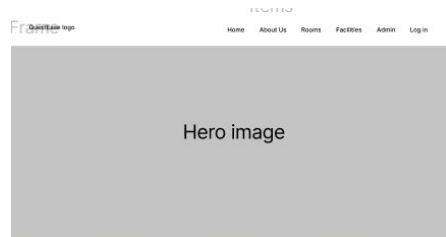
<https://supabase.com/dashboard/project/zpimkgxflffiooxowet/sql/>

Use case diagram

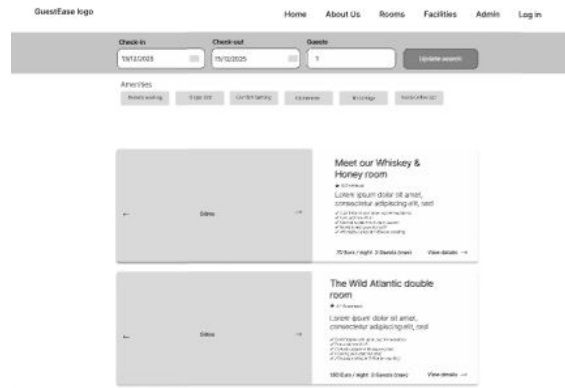


Analysis & Design

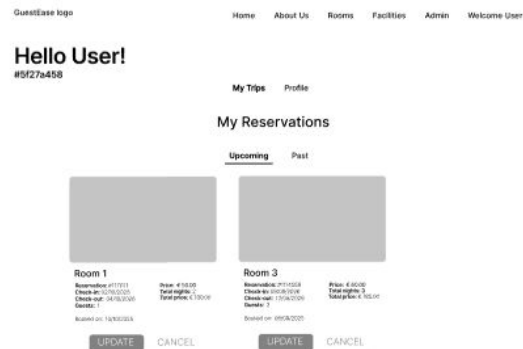
Figma wireframes (some examples)



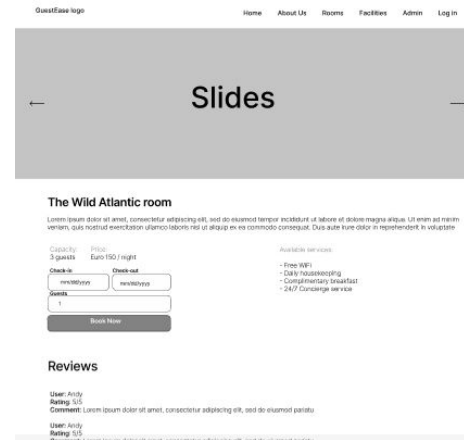
Homepage



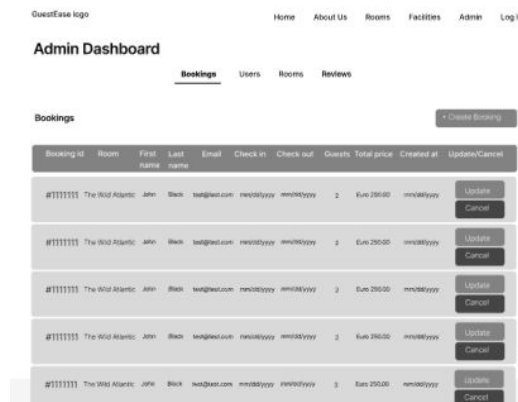
Search results page



My Trips page



Room Details page



Admin dashboard

<https://www.figma.com/design/2FIwh7ESiHDX6UvxikZiUH/Final-Project?node-id=0-1&p=f&t=7Q4Z32fDIGVeayat-0>



Development Process

Sprint 1 – GuestEase App Foundation

- Project plan on Trello, architecture set-up, gitHub repo, wireframes, static pages build, first Supabase connection with a static page (Rooms).

Sprint 2 – Core ‘User/Guest’ Features

- Auth (Login/Signup), Profile & My Trips, Search Form, Search Results, and Room Details page, booking creation.

Sprint 3 – User(Guest) Backend Support & Payments

- Booking + profile CRUD, Stripe payment & booking confirmation flow integrated with Supabase.

Sprint 4 – Admin system & Guest email notifications

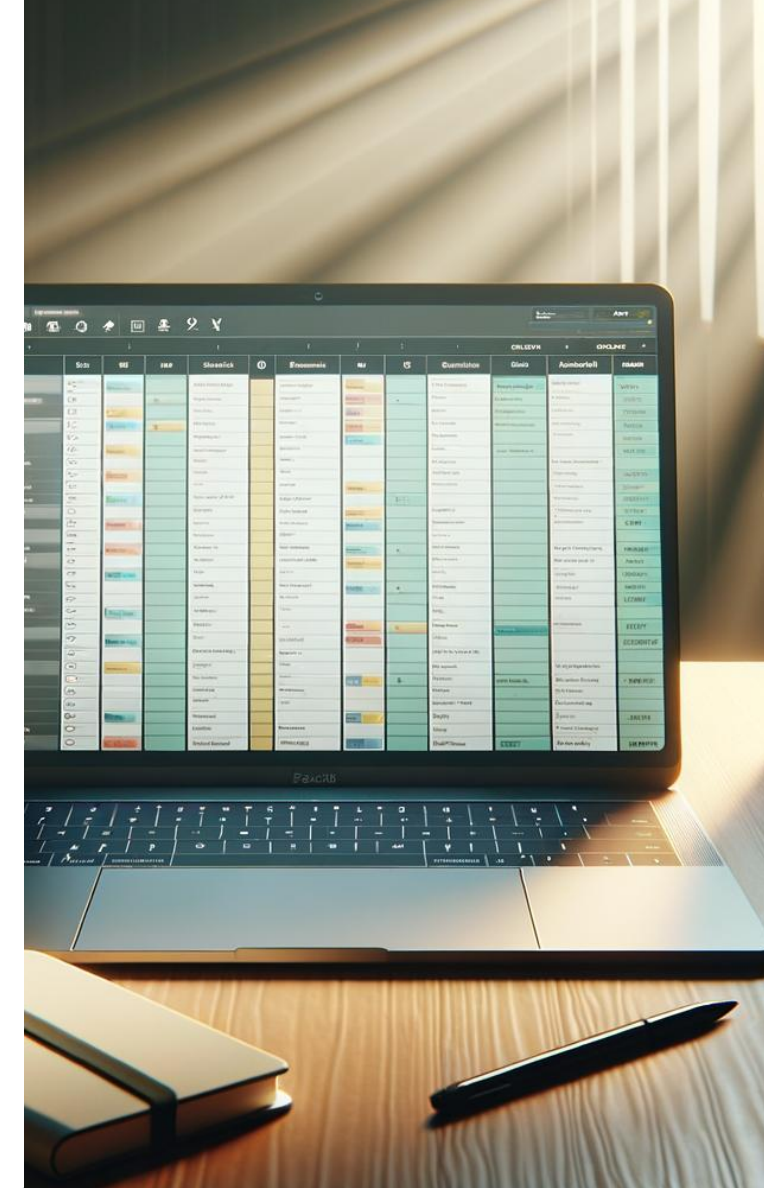
- Admin dashboard with CRUD functionalities (rooms, bookings, users, reviews), automated guest emails, expanded backend and Supabase tables/SQL logic.

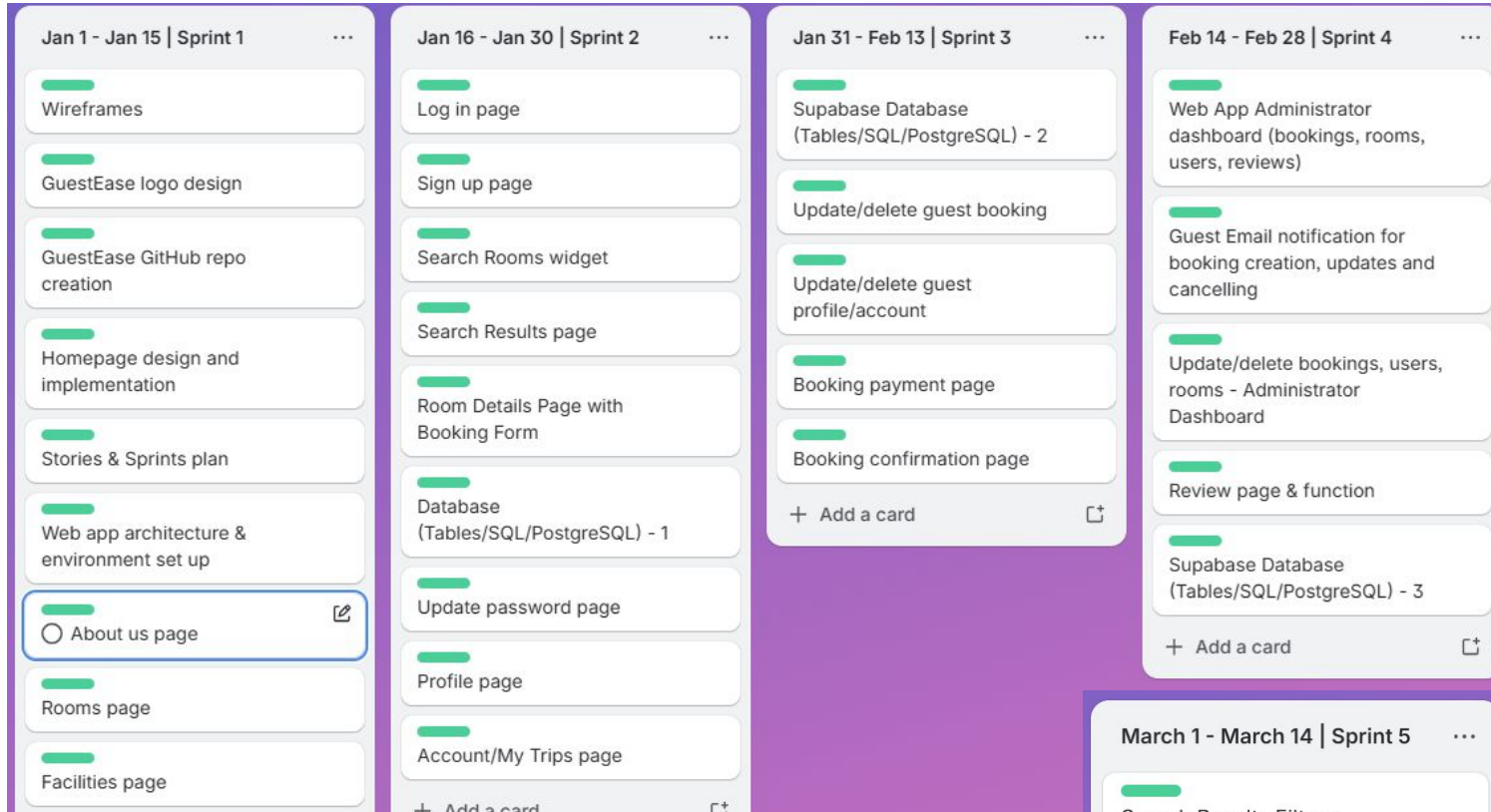
Sprint 5 – Enhancements & Deployment

- Search filters, admin email notifications, landing page, full deployment + testing.

Sprint 6 – Finalization

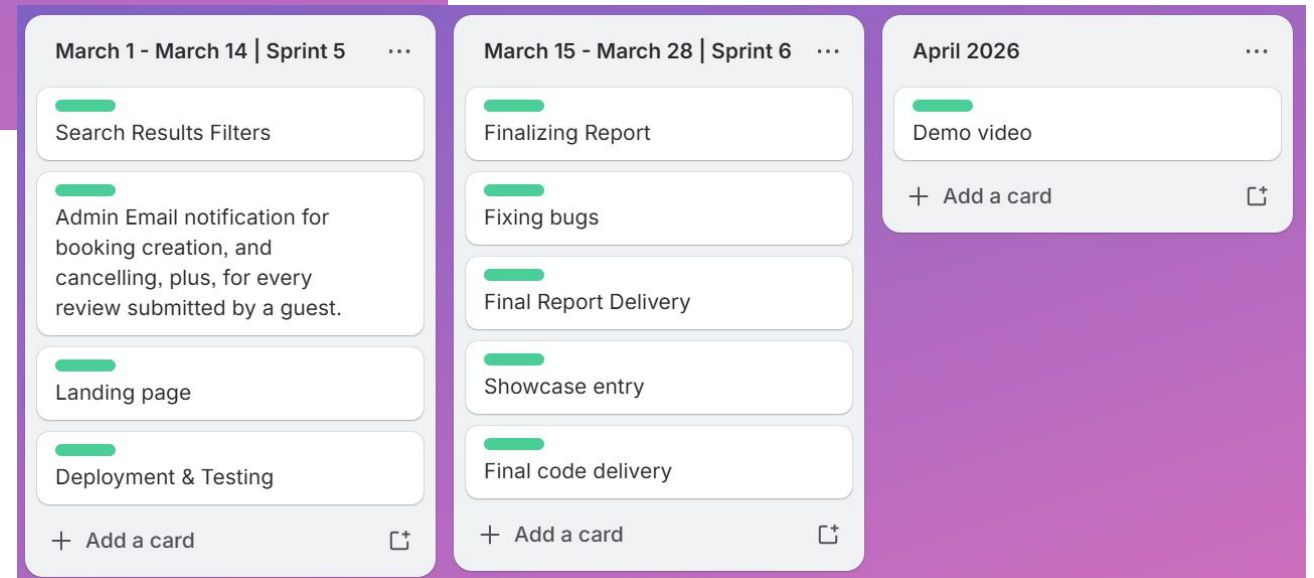
- Finalized documentation, bug fixes, final report, showcase submission, project delivery.





The GuestEase Trello board

<https://trello.com/b/ZYkH5dya/my-trello-board>



Development Process

Key Features Implemented

User Role-based Access

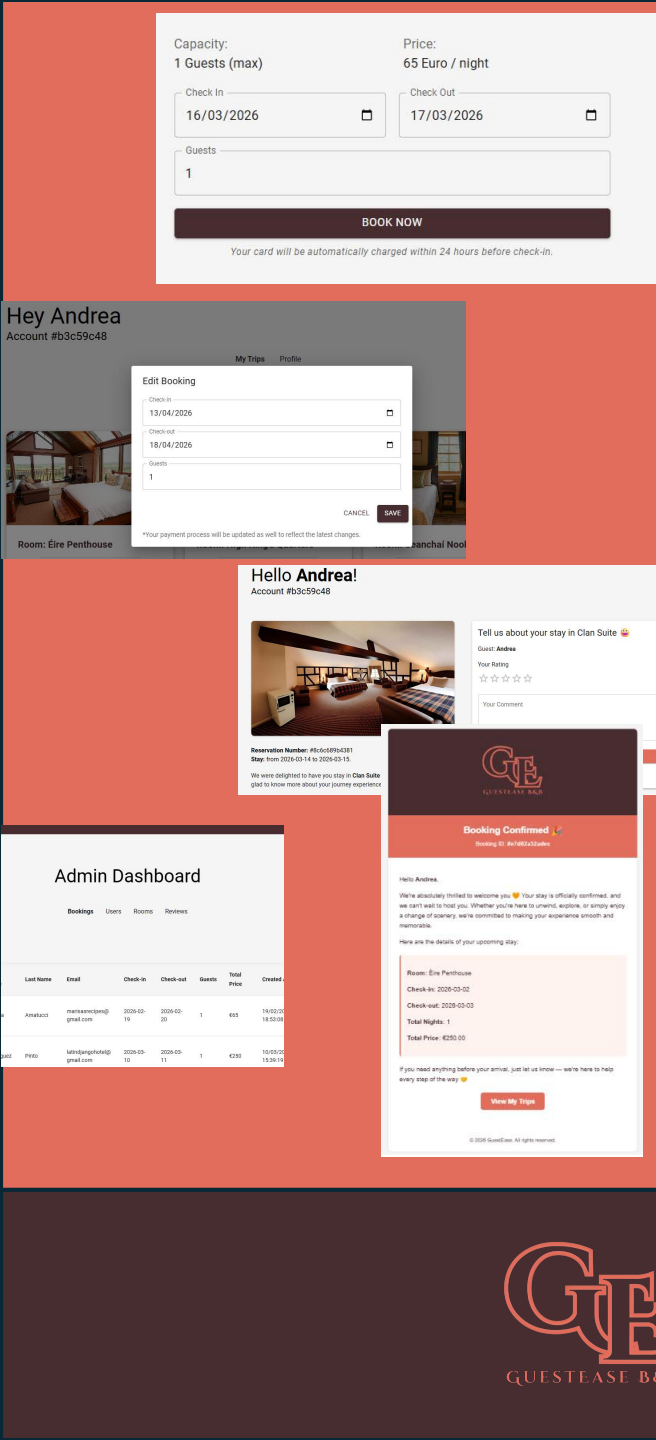
Guest and **Admin** roles with secure authentication; admins can create users

Booking system

Browse rooms by date/capacity, full CRUD, real-time availability, error-free workflow

Admin dashboard

Manage rooms, bookings, users, and view guest reviews



Key Features Implemented

Email notifications

CRUD bookings, payments, reviews, user created by admin and welcome emails

Stripe Payments

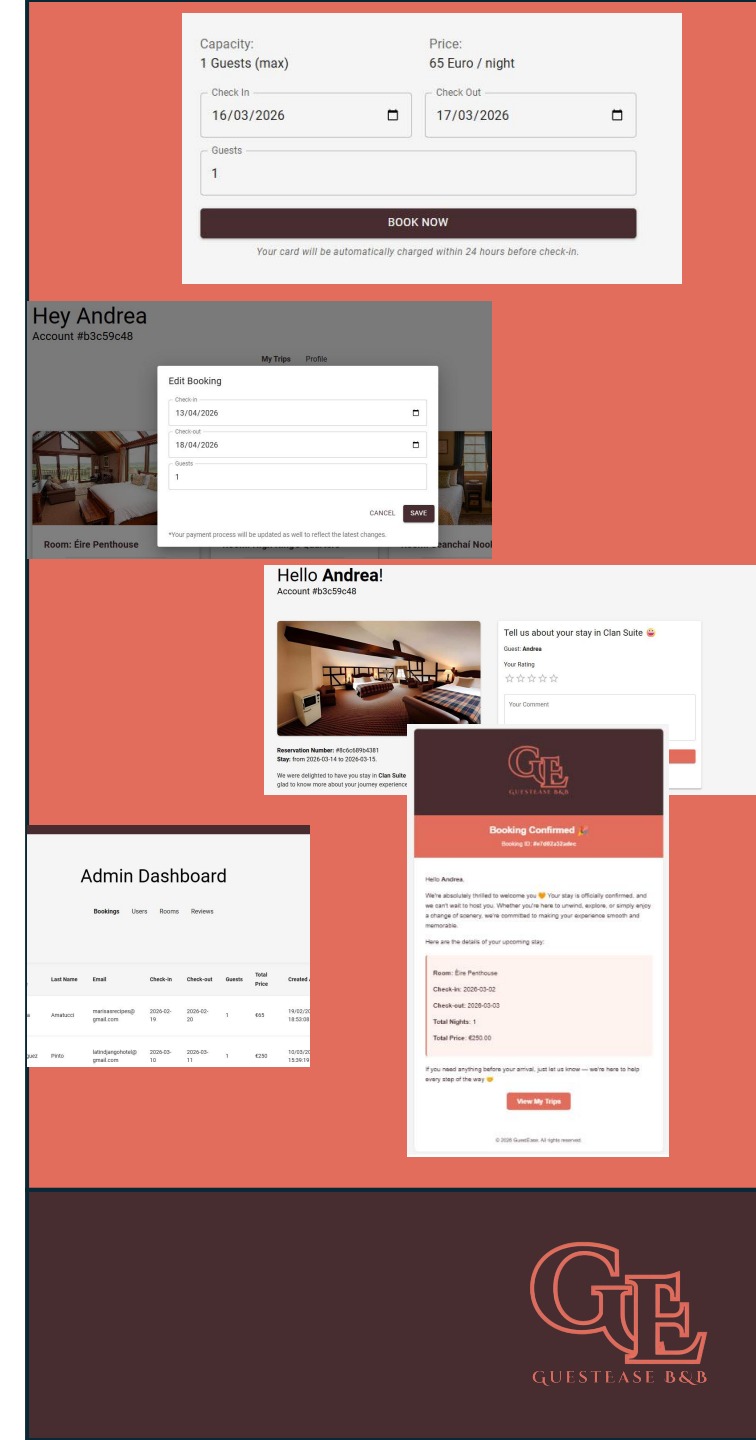
Store payment method, process payments 24h before check-in and handle refunds

Guest Reviews

Guests submit reviews, admin receives email notifications, viewable in dashboard

Responsive UI

Mobile-friendly and optimized across devices



Challenges & Solutions

- **Auth.users & Profiles Sync:**
Synced Supabase auth with profiles table using PostgreSQL triggers
- **Room Data Errors:**
Fixed invalid dates input issue by conditionally enabling queries
- **Booking Logic Issues:**
Resolved date format bug in price calculation using proper Date conversion
- **Data Validation Improvements:**
 - Prevented duplicate email sign-ups via custom DB function
 - Stopped booking overlaps with availability checks + UI validation
- **Frontend Stability:**
Fixed React hook-order error by restructuring component logic
- **Media Handling:**
Refactored image upload into custom hook for consistent rendering
- **Payments & Cron Fixes:**
Corrected Stripe cron failures caused by date format mismatch
- **Stripe Refund System:**
Added payments data storage to 'bookings' table to support reliable cancellations/refunds
- **External Limitations:**
Resend email restricted to one recipient (free tier requires domain for production)

```
no. serve(async () => {  
  /**  
   * A fix that was needed:  
   * check_in is a DATE column, not a TIMESTAMPTZ  
   * We must compare date to date, otherwise  
   * (e.g. check_in = "2026-02-14") won't  
   * runs later in the day.  
   * https://stackoverflow.com/questions/47061111  
   */  
  const today = now.toISOString().split("T")[0]  
  const tomorrow = in24h.toISOString().split("T")[0]  
  
  /**  
   * 1. Fetch bookings with check-in within  
   *    AND not yet charged.  
   * https://supabase.com/docs/reference/javascript  
   * Compare DATE strings instead of timestamps  
   * Limit to 5 booking to avoid Supabase RLS error  
   */  
  const { data: bookings, error } = await supabase  
    .from("bookings")  
    .select("*")  
    .lte("check_in", tomorrow)  
    .gte("check_in", today)  
    .eq("charged", false)  
    .limit(5);
```

Challenges & Solutions

Auth.users & Profiles table users Sync:

GuestEase / main PRODUCTION Connect Feedback Search... K

public.bookings public.rooms public.profiles public.reviews public.user_payment_methods +

Filter by id, email, first_name... or ask AI

Sort RLS policies Index Advisor Enable Realtime Role postgres Insert

id	email	first_name	last_name	country	zip_code	role	created_at	stripe_customer_id
04537395-0de2-428a-ac5e-fdd1e...	marisasrecip@gmail.com	Marisa	Amatucci	Italy	64016	guest	2026-02-06 17:04:48.579+00	cus_U0dbSQyhuV8KIO
04eb3ce4-3550-4887-a835-e947...	andrea.nardinocchi76@gmail.com	Sarah	Fitzpatrick	Ireland	T12F8R9	admin	2026-02-25 22:43:46.972172+0	cus_U2who4CKNDIbbL
09bdf790-7e34-41e5-9f4c-2e2521...	oviajantepelomundo76@gmail.com	Pedro	Conceção	Portugal	T12Y2NE	guest	2026-03-08 20:37:55.576864+0	cus_U729dGdHu0bpSm

Supabase 'Profiles' table on <https://supabase.com/dashboard/project/xxxxxxxxxxxxxx/editor/35435?schema=public>

GuestEase / main PRODUCTION Connect Feedback Search... K

Users

Q Email address Search by email All columns Sorted by user ID Add user

	UID	Display name	Email	Phone	Providers	Provider type	Created at	Last sign in at
☐	04eb3ce4-3550-4887-a835-e9476827fe81	Sarah Fitzpatrick	andrea.nardinocchi76@gmail.com	-	Email	-	Wed 25 Feb 2026 22:43:46 GMT+0000	Sun 22 Mar 2026 12:...
☐	cdcf4d45-5d09-4a0d-bdd1-af9e4653bc48	Andrea Nardinocchi	andymarnard@gmail.com	-	Email	-	Sun 22 Mar 2026 12:56:50 GMT+0000	-

Supabase 'Authentication' section on <https://supabase.com/dashboard/project/xxxxxxxxxxxxxx/auth/users>

```
supabase.com/docs/guides/auth/managing-user-data#using-triggers

1 -- inserts a row into public.profiles
2 create function public.handle_new_user()
3 returns trigger
4 language plpgsql
5 security definer set search_path = ''
6 as $$
7 begin
8   insert into public.profiles (id, first_name, last_name)
9   values (new.id, new.raw_user_meta_data -> 'first_name', new.raw_user_meta_data -> 'last_name');
10  return new;
11 end;
12 $$;
13
14 -- trigger the function every time a user is created
15 create trigger on_auth_user_created
16 after insert on auth.users
17 for each row execute procedure public.handle_new_user();
```

<https://supabase.com/docs/guides/auth/managing-user-data#using-triggers>

```
-- Trigger function to handle new users
-- This automatically inserts a profile row whenever a new auth user is created.
-- Supabase docs (trigger pattern):
-- https://supabase.com/docs/guides/auth/managing-user-data#using-triggers
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
BEGIN
  INSERT INTO public.profiles (
    id,
    email,
    first_name,
    last_name,
    country,
    zip_code
  )
  VALUES (
    NEW.id,
    NEW.email,
    NEW.raw_user_meta_data->'first_name',
    NEW.raw_user_meta_data->'last_name',
    NEW.raw_user_meta_data->'country',
    NEW.raw_user_meta_data->'zip_code'
  )
  ON CONFLICT (id) DO NOTHING;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- TRIGGER ON auth.users
-- Runs the function handle_new_user() after each new user signup.
-- https://supabase.com/docs/guides/auth/managing-user-data#using-triggers
DROP TRIGGER IF EXISTS on_auth_user_created ON auth.users;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW
EXECUTE FUNCTION public.handle_new_user();

-- ENABLE ROW LEVEL SECURITY
-- https://supabase.com/docs/guides/auth/row-level-security
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
```

Challenges & Solutions

Booking overlap bug when editing booking dates

account/mytrips

Home About us Rooms

My Trips Profile

Edit Booking

Check-in
01/30/2026

This room is already booked for these dates

Check-out
02/27/2026

This room is already booked for these dates

Guests: 1
Price: €200.0
Total nights: 1
Total price: €200.0

CANCEL SAVE

src/page/accountMyTripsPage.tsx

```
42 const EditBookingDialog: React.FC<EditBookingDialogProps> = ({
43   const checkoutDate = new Date(localBooking.check_out),
44   const dateError = checkInDate >= checkOutDate;
45
46   const { data: availableRooms } = useAvailableRooms(
47     localBooking.check_in,
48     localBooking.check_out,
49     localBooking.guests,
50   );
51   /**
52    * Fetching the booking id to exclude when checking the room availability for updates.
53    * If we do not exclude it, the modal will show the overlapError at any time.
54    * We want it to only show when there is an actual overlap with other bookings.
55    */
56   localBooking.id,
57 );
```

src/components/editBookingDialog.tsx

```
FROM public.rooms AS r
WHERE
-- Ensure room can accommodate the requested number of guests
r.capacity >= guests
-- Exclude rooms that overlap with an existing booking
-- The "&&" operator checks whether two ranges overlap.
-- PostgreSQL docs: Table 9.58 (range operators) https://www.postgresql.org/docs/current/functions-range.html
AND NOT EXISTS (
  SELECT 1
  FROM public.bookings AS b
  WHERE b.room_id::uuid = r.id
  AND (exclude_booking_id IS NULL OR b.id != exclude_booking_id)

  -- Compare the booking's stored daterange (b.period)
  -- with the requested range created by daterange($1, $2, '{}')
  -- '{}' means:
  -- inclusive lower bound (check_in)
  -- exclusive upper bound (check_out) so that when a check out exists a check in can exist too for the same date
  AND b.period && daterange($1, $2, '{}')
);
```

Supabase function get_available_rooms

```
// Here we fetch the room id of our booked room to check its availability
const roomIsAvailable =
  availableRooms?.some((r: any) => r.id === room.id) ?? true;

// If the room is not available, we raise the overlapError
const overlapError = !roomIsAvailable;
```

```
/**
 * By disabling the button in case of any errors, we prevent overlapping
 * and invalid date selections.
 */
disabled={overlapError || dateError}
sx={{
  backgroundColor: "#472d30",
  "&:hover": { bgcolor: "#e26d5c" },
}}
>
  Save
</Button>
```

src/components/editBookingDialog.tsx

```
bookings Multiuse updates rooms function
RETURNS TABLE (
)
LANGUAGE sql
SECURITY DEFINER
SET search_path = public
AS $$
-- This function is executed with SECURITY DEFINER,
-- meaning it runs with the privileges of the function owner rather
-- This pattern is recommended for RPC functions that need control
-- to tables protected by Row Level Security (RLS).
-- https://supabase.com/docs/guides/database/functions
SELECT
  r.id,
  r.name,
  r.description,
  r.capacity,
  r.price,
  r.amenities,
  r.images,
  r.created_at
FROM public.rooms AS r
WHERE
-- Ensure room can accommodate the requested number of guests
r.capacity >= guests
-- Exclude rooms that overlap with an existing booking
-- The "&&" operator checks whether two ranges overlap.
-- PostgreSQL docs: Table 9.58 (range operators) https://www.po
AND NOT EXISTS (
  SELECT 1
  FROM public.bookings AS b
  WHERE b.room_id::uuid = r.id
  AND (exclude_booking_id IS NULL OR b.id != exclude_booking_id)

  -- Compare the booking's stored daterange (b.period)
  -- with the requested range created by daterange($1, $2, '{}')
  -- '{}' means:
  -- inclusive lower bound (check_in)
  -- exclusive upper bound (check_out) so that when a check
  AND b.period && daterange($1, $2, '{}')
```

AI & GuestEase

- **Creative support**

Some AI tools (Copilot, DeepAI Image-Generator) have been used to create room descriptions, amenities, images and other content throughout the web app pages. Prompts such as “*Create rooms with Irish-inspired names and tailored amenities...*” or “*helped speed up content creation.*”

- **Architecture, Best-Practice Validation & Problem-Solving** (Some examples below listed)

Discussed how to run Stripe logic server-side

“...expose server-side endpoints responsible for creating payment intents, validating sessions, and handling webhooks. Client components should only request these values and never interact with Stripe directly.” - Copilot

how to structure the booking CRUD

architecturally on the backend and why

“... clean approach is to expose dedicated endpoints for create, update, and delete operations, each performing validation and permission checks before interacting with the database...” - Copilot

problem-solving support “*...Rendered more hooks than during the previous render*” (Console log error)...

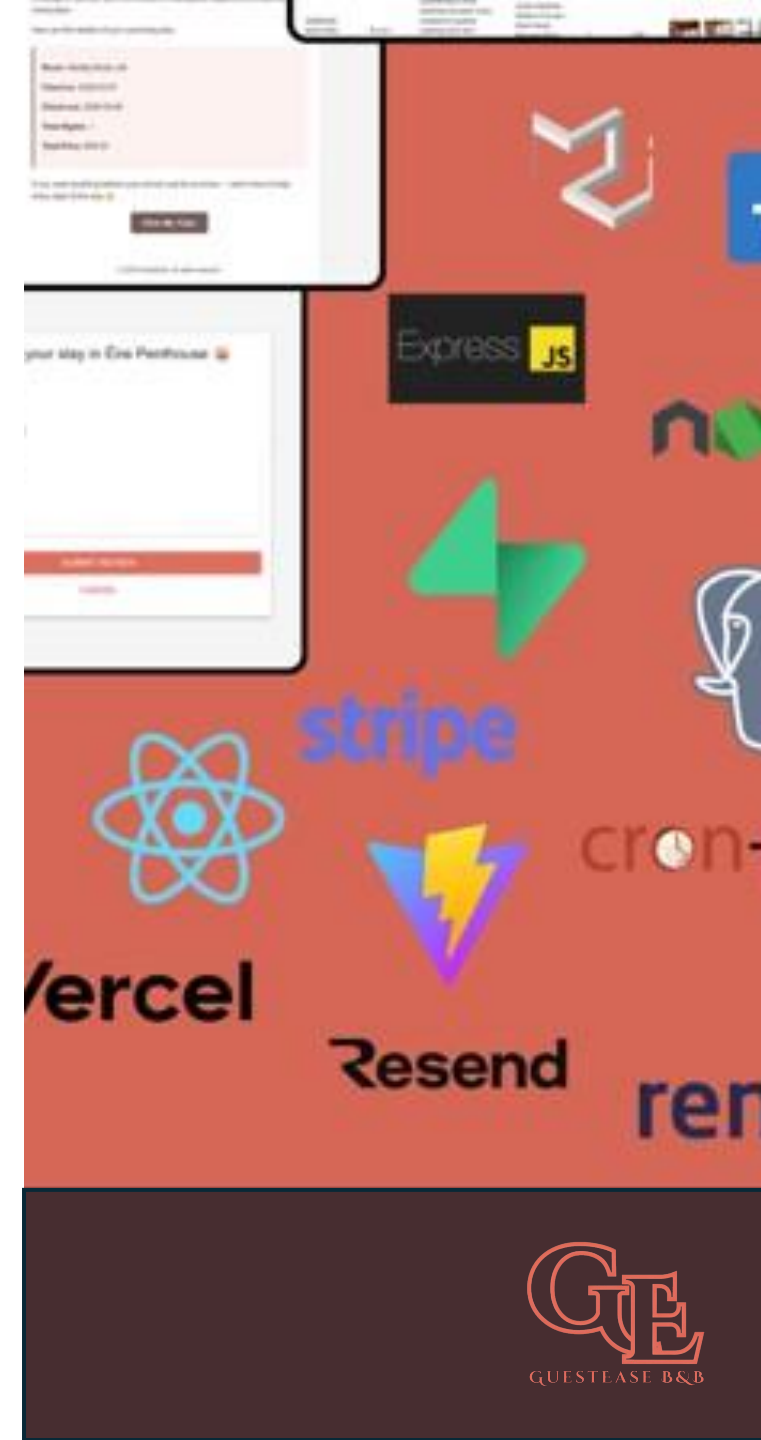
“...your component is calling hooks in a different order between renders. React requires that hooks run in the exact same sequence every time a component renders...” - Copilot



What I Learned

Gained hands-on experience with **Supabase SQL**, **PostgreSQL**, **Stripe**, **Resend**, **cron-job.org**, and strengthened **React**, **Material UI**, **Node.js**, **Express**....

...delivered a **functional booking platform** with core CRUD features, **admin dashboard**, **Stripe payments**, and **automated email workflows**.



Future Development

- Introducing a **loyalty reward system** for guests with points, discounts, and promos
- Expanding GuestEase into a **multi-property platform** beyond the 8-room B&B prototype
- Adding **property filters** and **map-based room locations** for better user experience
- Enhancing **backend security**: enforce RLS and validate admin roles
- Improving **payment flow**: store payment methods per booking instead of a single default card
- Continuing **refining the app** to make it production-ready and scalable

```
const elements = useElements();

// useState to load the modal
const [loading, setLoading] = useState(false)
// useState for the postal code field
const [postalCode, setPostalCode] = useState('')

// useState to show error message to show why
const [error, setError] = useState<string | null>(null)

/**
 * Handles form submission and triggers the payment flow
 * https://stripe.com/docs/payments/accept-a-payment
 * https://stackoverflow.com/questions/74806100/stripe-js-error-card-element-not-found
 */
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setError(null);

  // https://docs.stripe.com/sdks/stripejs-quickstart
  if (!stripe || !elements) return;

  setLoading(true);

  const card = elements.getElement(CardElement);

  if (!card) {
    setError("Card element not found");
    setLoading(false);
  }
}
```

Questions

Book a room now...
...and have a nice stay!
;-)

